

Foundations of Artificial Intelligence

Part II: Problem Solving and Search

Department of Computer Science

Lecture Roadmap

1. From Problem Formulation to Search
2. Uninformed Search
3. Informed Search
4. Heuristics and Optimality

Learning Objectives

By the end of this lecture, you should be able to:

- Formulate an AI task as a search problem over a state space.
- Compare uninformed strategies: BFS, DFS, and Uniform-Cost Search.
- Explain informed search with Greedy Best-First and A*.
- Use heuristic properties (admissibility and consistency) to reason about optimality.
- Choose a strategy based on completeness, optimality, time, and space trade-offs.

From Problem Formulation to Search

Why Search Matters in AI

Problem-solving agents follow a simple loop:

Formulate → Search → Execute

- **Formulate:** Convert the task into states, actions, and goals.
- **Search:** Find an action sequence from initial state to a goal state.
- **Execute:** Apply the sequence in the environment.

Core idea from the book: intelligence emerges from representation + algorithm.

State Space Representation

A search problem is specified as:

$$\langle S, A, s_0, G, c \rangle$$

- S : set of states.
- $A(s)$: actions available in state s .
- s_0 : initial state.
- G : goal test or set of goal states.
- $c(s, a, s')$: step cost function.

A solution is a path from s_0 to a goal; an **optimal** solution has minimum total path cost.

Search Tree vs State Graph

Search Tree View

- Nodes are paths.
- Same state may appear many times.
- Can become infinite with loops.

State Graph View

- Nodes are unique states.
- Graph-search tracks visited states.
- Usually reduces repeated work.

Practical rule: Prefer graph search unless memory constraints are severe.

Running Example: Route Finding

Component	Example
States	Cities on a map
Actions	Drive to an adjacent city
Initial state	Milan
Goal test	Reach Rome
Step cost	Road distance in km

This classic setup lets us compare BFS, DFS, Uniform-Cost, Greedy, and A* on the same problem.

Uninformed Search

Uninformed Search: What It Knows

Uninformed (blind) search uses only:

- successor function,
- goal test,
- path cost definition (if relevant).

It has no estimate of how close a state is to the goal.

Main strategies in this lecture:

- Breadth-First Search (BFS)
- Depth-First Search (DFS)
- Uniform-Cost Search (UCS)

Breadth-First Search (BFS)

- Expands shallowest frontier node first (FIFO queue).
- Complete if branching factor is finite.
- Optimal when all step costs are equal.

Typical complexity (tree view):

$$\text{Time} = O(b^d), \quad \text{Space} = O(b^d)$$

where b is branching factor and d is depth of shallowest goal.

Strength: Reliable shallow solution. **Weakness:** Very high memory use.

Depth-First Search (DFS)

- Expands deepest frontier node first (stack or recursion).
- Space-efficient because it stores one path plus siblings.
- Not optimal; can be incomplete in infinite-depth spaces.

Typical complexity (tree view):

$$\text{Time} = O(b^m), \quad \text{Space} = O(bm)$$

where m is maximum depth.

Strength: Low memory footprint.
direction.

Weakness: Can go deep in wrong

Uniform-Cost Search (UCS)

- Expands node with smallest path cost $g(n)$ first.
- Implemented with a priority queue.
- Equivalent to Dijkstra-style expansion on positive-cost edges.

Guarantees:

- Complete if each step cost is at least some $\epsilon > 0$.
- Optimal for general nonnegative step costs.

Key difference from BFS: UCS optimizes path cost, not depth.

Uninformed Strategy Comparison

Strategy	Complete?	Optimal?	Time	Space
BFS	Yes (finite b)	Yes (equal costs)	Exponential in d	Exponential in d
DFS	No (infinite depth)	No	Exponential in m	Linear in depth
UCS	Yes ($c \geq \epsilon$)	Yes	Exponential in C^*/ϵ	Similar to time

Informed Search

From Blind to Informed Search

Informed search adds a heuristic estimate:

$$h(n) \approx \text{estimated cost from } n \text{ to a goal}$$

Better estimates usually mean fewer expansions.

Two central strategies:

- **Greedy Best-First:** expand smallest $h(n)$.
- **A*:** expand smallest $f(n) = g(n) + h(n)$.

Greedy Best-First Search

- Chooses node that appears closest to goal by $h(n)$.
- Often very fast with good heuristics.
- Can be misled into long or costly detours.

Evaluation:

$$f(n) = h(n)$$

Not generally optimal. Not always complete in graph variants without safeguards.

A* balances cost-so-far and estimated cost-to-go:

$$f(n) = g(n) + h(n)$$

- $g(n)$: exact path cost from start to n .
- $h(n)$: heuristic estimate from n to goal.

Intuition:

- Greedy alone over-trusts h .
- UCS alone ignores goal direction.
- A* combines both signals.

Heuristics and Optimality

Heuristic Quality Conditions

Admissible heuristic:

$$0 \leq h(n) \leq h^*(n)$$

It never overestimates true remaining cost $h^*(n)$.

Consistent heuristic (monotone):

$$h(n) \leq c(n, a, n') + h(n')$$

for every edge (n, a, n') .

Consistency implies admissibility (with $h(\text{goal}) = 0$) and gives nondecreasing f -values along paths.

When Is A* Optimal?

For tree search, A* is optimal if h is admissible.

For graph search, A* is optimal with a consistent heuristic and proper closed-list handling.

Why (high-level):

1. A* always expands the node with smallest f .
2. For admissible h , $f(n)$ never overestimates best solution through n .
3. Therefore, the first goal removed from frontier has cost C^* .

Designing Heuristics in Practice

Reliable methods:

- **Relaxed problems:** remove constraints and solve easier version.
- **Abstractions:** compress state description while preserving lower bounds.
- **Pattern databases:** precompute exact costs for abstract subproblems.

Quality trade-off:

more accurate $h \Rightarrow$ fewer expansions, often more computation per node

Example Heuristic: Straight-Line Distance

In route planning, let $h(n)$ be straight-line distance to destination.

- Usually admissible: roads cannot be shorter than straight-line geometry.
- Often consistent due to triangle inequality.

Result:

- Greedy becomes fast but may pick costly routes.
- A* with this heuristic is both efficient and optimal (under assumptions above).

Common Pitfalls and Fixes

- **Pitfall:** Confusing state with node/path record.
- **Fix:** Track parent, action, and path cost per node explicitly.
- **Pitfall:** Using heuristic that overestimates and expecting optimality.
- **Fix:** Validate admissibility/consistency on representative transitions.
- **Pitfall:** Ignoring repeated states.
- **Fix:** Use graph-search with reached/closed set and best-cost updates.

From Search to Algorithmic Intelligence

Why this topic is foundational:

- You model the world as formal states and transitions.
- You reason about guarantees: completeness and optimality.
- You make principled trade-offs: speed, memory, and solution quality.
- You design heuristics that inject domain knowledge into generic algorithms.

This is where intelligent behavior becomes an engineering discipline.

Key Takeaways

1. State-space representation determines what search can do.
2. BFS, DFS, and UCS differ mainly in frontier ordering and guarantees.
3. Greedy is goal-directed but not reliably optimal.
4. A* is optimal with admissible/consistent heuristics and correct graph handling.
5. Better heuristics can drastically reduce search effort.

Reference Reading for This Part

- **Russell & Norvig (3rd/4th ed.):** Problem-solving and search chapters.
- Focus sections: problem formulation, tree/graph search, uninformed and informed search, heuristic properties.

Suggested self-check:

1. Can you write $\langle S, A, s_0, G, c \rangle$ for a new domain?
2. Can you justify whether a heuristic is admissible?
3. Can you predict whether an algorithm is complete/optimal in that domain?

Questions?